

01. Your task for this exercise will be to develop a Prolog predicate called `roman2arabic/2` to translate from Roman to the usual Arabic numerals. Examples:

```
?- roman2arabic(XXI, Number).  
Number = 21  
true
```

```
?- roman2arabic(MCMXCIX, Number).  
Number = 1999  
true
```

Before you start thinking about how to implement this, remind yourself how Roman numerals work exactly, using appropriate online resources (such as Wikipedia).

The key idea we will use to approach this problem is to work with an intermediate representation consisting of lists of numbers corresponding to the symbols used for Roman numerals. For example, the number XXI will be represented as the list [10, 10, 1]: each number corresponds to one of the symbols, and the sum of the numbers is 21, the Arabic equivalent of XXI. In fact, because Roman numerals make use of the so-called subtractive notation (writing, for instance, IX rather than VIII for 9), it will get a little more complicated than this. For example, rather than representing XIX as [10, 1, 10], the sum of which would be 21 rather than the 19 we need, we will represent XIX as [10, 9]. That is, sometimes two Roman symbols are represented by a single number in the list (e.g., the I and X together are represented by the 9). Follow these steps to implement `roman2arabic/2`:

- (a) Implement a predicate `symbol/2` to retrieve for each symbol used in the Roman system the corresponding numerical value. You should cover I, V, X, L, C, D, and M. Example:

```
?- symbol(X, Value).  
Value = 10  
true
```

- (b) Now implement a predicate `symbols2numbers/2` to translate a given list of symbols to the corresponding list of its numerical values. Your initial version of the predicate should ignore the complication of the subtractive notation and simply translate each symbol separately. Examples:

```
?- symbols2numbers([M, M, X, V], Values).  
Values = [1000, 1000, 10, 5]  
true
```

```
?- symbols2numbers([X, L, I, I], Values).  
Values = [10, 50, 1, 1]  
true
```

Once this is working, add a further rule to your implementation to handle the subtractive notation. For the last example above, you should now get the following output (because XL represents the number 40, not the two numbers 10 and 50):

```
?- symbols2numbers([X, L, I, I], Values).
Values = [40, 1, 1]
true
```

To achieve this, as you go through the input list, you need to always first scan the two initial symbols in the list. Whenever the first of these two symbols have a value that is less than the value of the second symbol, you know that you have encountered an instance of the subtractive notation and you can calculate the corresponding single number.

- (c) Now write a predicate `sum/2` to compute the sum of a list of numbers provided in the first argument position. Example:

```
?- sum([1, 2, 3, 4, 5], Sum).
Sum = 15
true
```

- (d) Now put everything together to implement `roman2arabic/2`. You will need to be able to break up an atom such as XXI into a list of characters (e.g., [X, X, I]). Check the reference manual of your Prolog system for how to do that.

02. One of the major news stories involving AI in the early 21st century has been about IBM Watson, a computer program that successfully competed in the American television game show Jeopardy! in 2011, beating the very best human contestants. Another famous television game show is the British Countdown (also known as Cijfers en Letters in the Netherlands and as Des Chiffres et des Lettres in France, where it had been broadcast first). The purpose of this exercise is to see whether we can win this one for AI as well. We will focus on the Letters Game of the Countdown show. In this game, we are given nine letters of the alphabet (possibly including some repetitions). The goal then is to construct the longest possible word from these letters. Your score is the length of your word (provided it is a valid word of the English language).

Your ultimate task is to write a Prolog predicate `topsolution/3` to play this game. When given a list of nine letters in the first argument position, it should return as good a solution as possible, consisting of a word of the English language that can be constructed from those letters, in the second argument position and the length of that word (i.e., the score) in the third argument position. Example:

```
?- topsolution([g,i,g,c,n,o,a,s,t], Word, Score).
Word = agnostic,
Score = 8
true
```

using the file **words.pl** and put it in the same directory as your program file. This is a list of a little over 350,000 of the most common words of the English language, from a to zyzzyva, presented as a sequence of facts, such as “**word(agnostic)**”, etc. Include the line `:- consult(words).` in your program to make these facts available to you. Then proceed as follows.

First, search your Prolog reference manual for a built-in predicate for decomposing an atom into a list of characters. Use it to implement a predicate `word_letters/2` for converting a word (i.e., a Prolog atom) into a list of letters. Example:

```
?- word_letters(hello, X).
X = [h, e, l, l, o]
true
```

As an aside, note that you can use this predicate to find words with 45 letters:

```
?- word(Word), word_letters(Word, Letters), length(Letters, 45).
Word = pneumonoultramicroscopicsilicovolcanoconiosis,
Letters = [p, n, e, u, m, o, n, o, u|...]
true
```

Second, write a predicate `cover/2` that, given two lists, checks whether the second list covers the first. That is, it checks whether every item that occurs *k* times in the first list also occurs at least *k* times in the second. Examples:

```
?- cover([a,e,i,o], [m,o,n,k,e,y,b,r,a,i,n]).
true
?- cover([e,e,l], [h,e,l,l,o]).
false
```

Third, write a predicate `solution/3` that, when given a list of letters as the first argument and a desired score as the third argument, returns a word covered by that list that would provide the given score (i.e., the length of which is equal to the desired score).

Example:

```
?- solution([g,i,g,c,n,o,a,s,t], Word, 3).
Word = act
true
```

Finally, implement `topsolution/3`. Document your program by showing how it performs for at least three different lists of letters. One of them should be `[y,c,a,l,b,e,o,s,x]`. This was one of the lists used in the edition of Countdown aired in Britain on 18 December 2002, when Julian Fell achieved the best overall score in the history of the show. He found the word cables, earning him a score of 6. Can your program beat the champion?